🏗️SFPPy Syntax: Multiple Inheritance for Custom Contact Scenarios 🍏⏩🍎

Introduction

SFPPy leverages **multiple inheritance** to define **custom food contact scenarios** with minimal effort. This approach allows users to seamlessly **combine different properties** (e.g., temperature, texture, and chemical affinity) into a single class, making simulation setup both intuitive and flexible.

This page explains how multiple inheritance enables:

- **Customization** 🎨: Easily define new contact conditions
- **Reusability** 🔄: Combine existing behaviors instead of rewriting code
- **Extensibility** 🚀: Adapt models to real-world packaging scenarios

1. Understanding Multiple Inheritance in SFPPy

1.1. Concept Overview 📚

In SFPPy, food contact conditions are defined as **classes** that inherit from multiple base classes. Each base class represents a **specific aspect** of the food contact condition, such as:

- **Storage conditions** (e.g., `ambient`, `hotfilled`, `chilled`)
- **Food texture, medium polarity...** (e.g., `fat`, `liquid`)
- **Food processing conditions** (e.g., `boiling`, `frying`)

- **Setoff conditions** (e.g., `stacked`)

By **combining** these classes, a new contact scenario can inherit properties from all relevant categories **without needing additional code**.

---

2. Defining Custom Contact Scenarios

2.1. Example: Defining a Stacked Hot-Filled Liquid Food Contact 🍲🔥

We can create a new **composite contact condition** by inheriting from existing classes:

```
from patankar.food import stacked, hotfilled, liquid, fat, ambient

class contact1(stacked, ambient):
    name = "1:setoff"
    contacttemperature = (20, "degC")
    contacttime = (3, "months")

class contact2(hotfilled, liquid, fat):
    name = "2:hotfilling"

class contact3(ambient, liquid, fat):
    name = "3:storage"
    contacttime = (6, "months")
```

Each class **inherits attributes** from its base classes:

✅ `stacked` → Defines a **setoff condition** (periodic boundary conditions) 📦 ✅ `ambient` → Assigns **storage at 20°C** 🌡️
✅ `hotfilled` → Models **hot-filling at 90°C** 🔥 ✅ `liquid`, `fat` → Define **food texture** 🥄

---

3. Applying Custom Scenarios to a Migration Simulation

3.1. Complete Example: Multi-Step Food Contact Simulation 🧪

```
from patankar.food import fat, ambient, hotfilled, stacked
from patankar.loadpubchem import loadpubchem
from patankar.layer import gPET, PP

# Define the migrant
m = loadpubchem('limonene')

# Define packaging structure
A = gPET(l=(20, "um"))
B = PP(l=(500, "um"), migrant=m, CP0=200)
ABA = A + B + A

# Instantiate food contact conditions
```

```
medium1 = contact1(contacttime=(4, "months"))
medium2 = contact2()
medium3 = contact3()

# Simulate migration
medium1 >> ABA >> medium1 >> medium2 >> medium3

# Combine results
sol123 = medium1.lastsimulation + medium2.lastsimulation + medium3.lastsimulation
sol123.plotCF()
```

🔹 Each **food contact step** maintains **unique properties** 📊 🔹 **Minimal code** needed to define new scenarios 🎯 🔹 Works seamlessly with **existing SFPPy simulations** ⚡

---

4. Benefits of Multiple Inheritance in SFPPy 🏆

4.1. Rapid Scenario Definition ⚡

- No need to manually define attributes like temperature and time 🚀
- Just **combine existing behaviors** via class inheritance 🎨

4.2. Easy Customization 🔧

- Modify only the **needed** properties while reusing the rest ✅
- Enables **fine-grained control** over each scenario 🎯

4.3. Improved Code Reusability 🔄

- **No redundant code**—reuse base classes instead 🏗️
- Multiple simulation scenarios can share the **same logic** 🔄

Conclusion 🎯

SFPPy's use of **multiple inheritance** makes it **easy to define custom contact conditions** with minimal code. By simply inheriting from **predefined classes**, users can construct **realistic and complex food contact scenarios** efficiently. 🚀
